

Paralelização de primos

A atividade atual se divide em duas:

1. Implementar um programa que conta quantos primos existem de 2 até N .
2. Sem alterar a lógica do programa, adicionar a diretiva `#pragma omp parallel for` para paralelizar o for principal.

Para essa atividade, considere que existe uma função chamada `is_prime`, $\mathbb{N} \Rightarrow \text{bool}$ que indica se um certo número i é primo ou não.

Com isso, observe que para calcular quantos primos existem de 2 até N , basta iterar no range de 2 até N , chamar `is_prime`, caso a função retorne true, adicione isso a um contador.

Entrada: $N : \mathbb{N}$

Variáveis: acumulador : $\mathbb{N} \leftarrow 0$

para cada i **em** $2..N$ **faça:**

se `is_prime( $i$ )` **então:**

 acumulador $\leftarrow$ acumulador + 1

fim para

Em C, isso pode ser alcançado da seguinte forma:

```
#include <omp.h>
#include <stdbool.h>
#include <stdio.h>
#include <stdlib.h>

bool is_prime(int n) {
    if (n == 2) {
        return true;
    }
    if (n < 2 || n % 2 == 0) {
        return false;
    }

    int k = n / 2;

    for (int i = 3; i < k; i++) {
        if (n % i == 0) {
            return false;
        }
    }

    return true;
}

int size = 1e6;
```

```

int main(void) {
    int cnt = 0;

    double inicio, fim;

    inicio = omp_get_wtime();

    omp_set_num_threads(12);
    for (int i = 2; i < size; i++) {
        if (is_prime(i)) {
            cnt++;
        }
    }

    fim = omp_get_wtime();

    printf("Elapsed wall time:  %f seconds\n", fim - inicio);
    printf("%d\n", cnt);

    return EXIT_SUCCESS;
}

```

Para fazer a segunda parte pedida, basta alterar o loop principal na main para o seguinte:

```

omp_set_num_threads(12);
#pragma omp parallel for
for (int i = 2; i < size; i++) {
    if (is_prime(i)) {
        cnt++;
    }
}

```

Com apenas isso, agora o openMP consegue dividir o loop entre as threads disponíveis.

Entretanto, todas as threads vão estar acessando a variável cnt, lendo o valor atual e incrementando um. Como existe mais de uma thread, onde pelo menos uma (nesse caso todas) estão tentando alterar enquanto as outras lêem para incrementar o valor, chegamos a uma condição de corrida. Em testes locais, as diferenças entre o resultado paralelo e o sequencial foram gritantes:

```

→ clang -O3 -fopenmp tasks/task5.c -lm
→ ./a.out
PARALLEL EXECUTION
Elapsed wall time:  5.116803 seconds
6050
→ clang -O3 -fopenmp tasks/task5.c -lm
→ ./a.out
SEQUENTIAL EXECUTION
Elapsed wall time:  25.491134 seconds
78498

```

Perceba que houve uma diferença gritante entre o resultado sequencial(74.498 primos encontrados) e o paralelo (6050 primos encontrados).

Essa discrepância acontece pois a operação de ler o valor atual e incrementar um, não é atômica. Ela é separada em dois momentos: leitura do valor atual e atualização do valor atual.

Então, se uma thread (T1) fez a operação de ler o valor atual, mas outra thread (T2) entrou na frente e fez a operação completa N vezes, quando T1 for atualizar o valor, ele vai ter perdido as informações adicionadas por T2, gerando um estado inconsistente. Para exemplificar, segue uma sequência de operações que exemplificam o problema:

Entradas T1: Thread, T2: Thread

Variáveis: $v : \mathbb{N} = 10$

1. T1 lê $v = 10$
2. T2 lê $v = 10$
4. T2 escreve 11 ($10 + 1$) em v
4. T2 lê $v = 11$
5. T2 escreve 12 ($11 + 1$) em v
6. T1 escreve 11 ($10 + 1$) em v

Pesquisando um pouco sobre, encontrei a diretiva `#pragma omp atomic`, que resolve esse problema realizando leituras e updates atômicos sobre a variável:

```
omp_set_num_threads(12);
#pragma omp parallel for
for (int i = 2; i < size; i++) {
    if (is_prime(i)) {
        #pragma omp atomic
        cnt++;
    }
}
```

E com isso, conseguimos ganhos expressivos de performance por ser paralelo, com a segurança na correteza da implementação:

```
→ hpc clang -O3 -fopenmp tasks/task5.c -lm
→ hpc ./a.out
SAFE PARALLEL EXECUTION
Elapsed wall time: 5.301734 seconds
78498
```

Note que o resultado da última execução é igual ao resultado sequencial.